

ORDER MANAGEMENT SYSTEM

NYKAA BEAUTY STORE MANAGEMENT

1.Database Design

The Order Management module of the Nykaa Beauty Store Management System is designed to store and manage customer orders and the products included in each order.

It stores:

- **Customer order details**
- **Order date**
- **Total order amount**
- **Order status**
- **Products ordered**
- **Quantity**
- **Unit price**

The main tables used for order management are:

- 1. NykaaOrder**
- 2. NykaaOrder_Item**

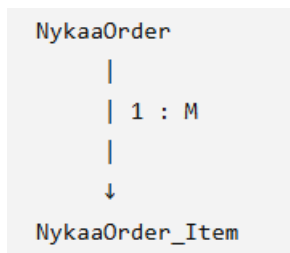
1.1 Orders Table

Attribute	Data Type	Key / Constraint	Description
Order_ID	NUMBER	Primary Key	Unique identification number of the order
Customer_ID	NUMBER	NOT NULL	Identifies the customer who placed the order
Order_Date	DATE	NOT NULL	Date on which the order was placed
Total_Amount	NUMBER(10,2)	—	Total amount of the order
Order_Status	VARCHAR2(20)	—	Status such as Delivered, Shipped, or Processing

1.2 Orders_Details Table

Attribute	Data Type	Key / Constraint	Description
Order_Item_ID	NUMBER	Primary Key	Unique identification number of the order item
Order_ID	NUMBER	NOT NULL	Identifies the related order
Product_ID	NUMBER	NOT NULL	Identifies the product in the order
Quantity	NUMBER	NOT NULL	Number of units ordered
Unit_Price	NUMBER(10,2)	NOT NULL	Price of one unit of the product

2. Table Relationships



NykaaOrder		NykaaOrder_Item
-----		-----
PK Order_ID	<----->	Order_ID
Customer_ID		PK Order_Item_ID
Order_Date		Product_ID
Total_Amount		Quantity
Order_Status		Unit_Price

Relationship Explanation

- One order can contain multiple order items.
- Each order item belongs to one specific order.
- Order_ID is the common linking attribute.
- NykaaOrder is the parent table.
- NykaaOrder_Item is the child table.

- Therefore, the relationship is One-to-Many (1:M).

3. Table Creation

3.1 Create Order Table

```
SQL> CREATE TABLE NykaaOrder (  
2     Order_ID NUMBER PRIMARY KEY,  
3     Customer_ID NUMBER NOT NULL,  
4     Order_Date DATE NOT NULL,  
5     Total_Amount NUMBER(10,2),  
6     Order_Status VARCHAR2(20)  
7 );
```

Table created.

3.2 Create Order_Details

```
SQL> CREATE TABLE NykaaOrder_Item (  
2     Order_Item_ID NUMBER PRIMARY KEY,  
3     Order_ID NUMBER NOT NULL,  
4     Product_ID NUMBER NOT NULL,  
5     Quantity NUMBER NOT NULL,  
6     Unit_Price NUMBER(10,2) NOT NULL  
7 );
```

Table created.

4. Order Data Insertion

```
SQL> INSERT INTO NykaaOrder
  2 VALUES (1001, 1, TO_DATE('18-09-2026','DD-MM-YYYY'), 2350.00, 'Delivered');

1 row created.

SQL>
SQL> INSERT INTO NykaaOrder
  2 VALUES (1002, 2, TO_DATE('19-09-2026','DD-MM-YYYY'), 1200.00, 'Shipped');

1 row created.

SQL>
SQL> INSERT INTO NykaaOrder
  2 VALUES (1003, 3, TO_DATE('20-09-2026','DD-MM-YYYY'), 1099.00, 'Processing');

1 row created.

SQL>
SQL> INSERT INTO NykaaOrder
  2 VALUES (1004, 4, TO_DATE('21-09-2026','DD-MM-YYYY'), 999.00, 'Delivered');

1 row created.

SQL>
SQL> INSERT INTO NykaaOrder
  2 VALUES (1005, 5, TO_DATE('22-09-2026','DD-MM-YYYY'), 850.00, 'Shipped');

1 row created.
```

5.Order_Details Data Insertion

```

SQL> INSERT INTO NykaaOrder_Item
  2 VALUES (1, 1001, 101, 3, 550.00);

1 row created.

SQL>
SQL> INSERT INTO NykaaOrder_Item
  2 VALUES (2, 1001, 102, 1, 700.00);

1 row created.

SQL>
SQL> INSERT INTO NykaaOrder_Item
  2 VALUES (3, 1002, 103, 2, 600.00);

1 row created.

SQL>
SQL> INSERT INTO NykaaOrder_Item
  2 VALUES (4, 1003, 104, 1, 1099.00);

1 row created.

SQL>
SQL> INSERT INTO NykaaOrder_Item
  2 VALUES (5, 1004, 105, 1, 499.00);

1 row created.

```

6.Order Modification Operations

6.1 Modify Order Amount

```

SQL> UPDATE NykaaOrder
  2 SET Total_Amount = 2500
  3 WHERE Order_ID = 1001;

1 row updated.

SQL> SELECT Order_ID, Total_Amount
  2 FROM NykaaOrder
  3 WHERE Order_ID = 1001;

ORDER_ID TOTAL_AMOUNT
-----
1001          2500

```

6.2 Modify Order Data

```
SQL> UPDATE NykaaOrder
  2  SET Order_Date = TO_DATE('23-09-2026', 'DD-MM-YYYY')
  3  WHERE Order_ID = 1001;
```

1 row updated.

```
SQL> SELECT Order_ID, Order_Date
  2  FROM NykaaOrder
  3  WHERE Order_ID = 1001;
```

ORDER_ID	ORDER_DAT
1001	23-SEP-26

7.Order_Details Modification

```
SQL> UPDATE NykaaOrder_Item
  2  SET Quantity = 4
  3  WHERE Order_Item_ID = 1;
```

1 row updated.

```
SQL> SELECT Order_Item_ID, Order_ID, Product_ID, Quantity, Unit_Price
  2  FROM NykaaOrder_Item
  3  WHERE Order_Item_ID = 1;
```

ORDER_ITEM_ID	ORDER_ID	PRODUCT_ID	QUANTITY	UNIT_PRICE
1	1001	101	4	550

8.Customer Order History Report

```

SQL> SELECT
  2     o.Customer_ID,
  3     o.Order_ID,
  4     o.Order_Date,
  5     p.Product_Name,
  6     oi.Quantity,
  7     oi.Unit_Price,
  8     (oi.Quantity * oi.Unit_Price) AS Item_Total,
  9     o.Total_Amount,
 10     o.Order_Status
 11 FROM NykaaOrder o
 12 LEFT JOIN NykaaOrder_Item oi
 13     ON o.Order_ID = oi.Order_ID
 14 LEFT JOIN Product p
 15     ON oi.Product_ID = p.Product_ID
 16 ORDER BY o.Order_Date DESC;

```

```

CUSTOMER_ID  ORDER_ID ORDER_DAT
-----

```

```

PRODUCT_NAME
-----

```

```

QUANTITY UNIT_PRICE ITEM_TOTAL TOTAL_AMOUNT ORDER_STATUS
-----

```

```

1      1001 23-SEP-26

```

```

4      550      2200      2500 Delivered

```

```

1      1001 23-SEP-26

```

QUANTITY	UNIT_PRICE	ITEM_TOTAL	TOTAL_AMOUNT	ORDER_STATUS
5	1005	22-SEP-26	850	Shipped
4	1004	21-SEP-26		
CUSTOMER_ID	ORDER_ID	ORDER_DAT		
PRODUCT_NAME				
QUANTITY	UNIT_PRICE	ITEM_TOTAL	TOTAL_AMOUNT	ORDER_STATUS
1	499	499	999	Delivered
3	1003	20-SEP-26		
1	1099	1099	1099	Processing
2	1002	19-SEP-26		
CUSTOMER_ID	ORDER_ID	ORDER_DAT		
PRODUCT_NAME				
QUANTITY	UNIT_PRICE	ITEM_TOTAL	TOTAL_AMOUNT	ORDER_STATUS
2	600	1200	1200	Shipped

6 rows selected.

9.Customer-wise Order Summary

```
SQL> SELECT
  2     o.Customer_ID,
  3     COUNT(DISTINCT o.Order_ID) AS Total_Orders,
  4     SUM(o.Total_Amount) AS Total_Amount
  5 FROM NykaaOrder o
  6 LEFT JOIN NykaaOrder_Item oi
  7     ON o.Order_ID = oi.Order_ID
  8 GROUP BY o.Customer_ID
  9 ORDER BY o.Customer_ID;
```

CUSTOMER_ID	TOTAL_ORDERS	TOTAL_AMOUNT
1	1	5000
2	1	1200
3	1	1099
4	1	999
5	1	850